

Boilerplate Structure

1. Put `LICENSE` file for your license
2. At `README.md` file for all command & explanation for everything, imagine some human don't know anything and want to use your project.

3. All source code put it in `src/`

4. Add `requirements.txt` files contains all used libraries and their versions like

```
fastapi==0.141.1
uvicorn[standard]==0.52.4
```

and you can import them via `pip install -r requirements.txt`

5. Add `.env` file UPPER_CASE keys for all your configurations & secret keys, copy `.env.example` from `.env` without adding any secret data and about the values have to be surrounded by double-quotation `"` not single `'` and don't contains `#` (some parsers consider it comment).

```
APP_NAME="Mini-RAG"
APP_VERSION="0.1"
FILE_ALLOWED_TYPES=["text/plain", "application/pdf"]
FILE_MAX_SIZE=10
```

6. Add `.gitignore` file for all files where you want to untracked by git and `.gitkeep` for uploading empty folders

7. Add folder `helpers/` containing `.env` handler like

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    APP_NAME: str
    FILE_ALLOWED_TYPES: list
    FILE_MAX_SIZE: int
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

def get_settings():
    return Settings()
```

8. Inside any folder create `__init__.py` file to collapsed importing chain like

```
from .enums.ResponseEnums import ResponseSignal
```

9. Add `assets/` contains data files like `API.yaml` & images & ..., can add `.gitignore` & `.gitkeep` files for improve tracking

10. Divide you Arch into MVC (Models Views Controllers), so create 3 dirs `Models/` everything related with data, `Views/` related with UI/UX, and `Controllers` related with logic and adding parent base inheriting class containing shared data and functions needed in any child controller

11. For all const data, define them @ Enum like

```
from enum import Enum
class ResponseSignal(Enum):
    FILE_VALIDATED_SUCCESS: str = "file_validate_successfully"
```

Code

- Make `main.py` file with minimal code, just define `app` and its routes like

```
from fastapi import FastAPI
from routes import base
app = FastAPI()
app.include_router(base.base_router)
```

- Inside `routes/` define `files.py` for each endpoint and add `schemes/` inside `routes/` to define transferred JSON objects like

```
.
├── base.py
├── data.py
├── __init__.py
└── schemes
    ├── data.py
    └── __init__.py
```

- Inside each `endpiont.py`

1. Define route with tag (for docs)

```
data_router = APIRouter(
    prefix='/api/v1/data',
    tags=['api_v1', 'data'],
)
```

2. Add logs logic by logging module

```
import logging
logger = logging.getLogger('uvicorn.error')
logger.error('Error while uploading file: ...')
```

3. If operation take data from project, define params with `Depends` fn in `fastapi` and make any endpoint `async` if needed like

```
@data_router.post('/upload/{project_id}')
async def upload_data(project_id: str, file: UploadFile,
                      app_settings: Settings = Depends(get_settings)):
    pass
```

4. make any operaiton return obj like `JSONResponse` to define status like

```
JSONResponse(
    status_code = status.HTTP_400_BAD_REQUEST,
    content = {
        'signal': ResponseSignal.FILE_UPLOAD_FAILED.value,
    }
)
```

1. FastAPI was built on Starlette so initially if loaded file size > 1MB stored on hard disk @ `/tmp/`, so to save RAM storage load chunks and this store them @ your place like

```
try:
    async with aiofiles.open(file_path, 'wb') as f:
        while chunk := await file.read(app_settings.FILE_DEFAULT_CHUNK_SIZE):
            await f.write(chunk)
except Exception as e:
    logger.error(f'Error while uploading file: {e}')
    return JSONResponse(
        status_code = status.HTTP_400_BAD_REQUEST,
        content = {
            'signal': ResponseSignal.FILE_UPLOAD_FAILED.value,
        }
    )
```

2. Create docker image for MongoDB @ `docker-compose.yml` like

```
services:
  mongodb:
    image: mongo:latest
    container_name: mongodb
    ports:
      - "27007:27017"
    volumes:
      - ./mongodb:/data/db
    networks:
      - backend
    restart: always
networks:
  backend:
```

3. Define configs in `.env` for MongoDB like

```
MONGODB_URL="mongodb://localhost:27007"
MONGODB_DATABASE="mini-rag"
```

4. You can install `Studio T3` for MongoDB
5. Install `motor` mongodb **asyncio** lib supporting via mongodb, don't forget add it @ `requirements.txt`
6. Prefer to use `os` lib to handle dir paths to be independent on OS.
7. Link mongodb + motor + fastapi like

```
from fastapi import FastAPI
from motor.motor_asyncio import AsyncIOMotorClient
from helpers.config import get_settings
app = FastAPI()
@app.on_event('startup')
async def startup_db_client():
    settings = get_settings()
    app.mongo_conn = AsyncIOMotorClient(settings.MONGODB_URL)
    app.db_client = app.mongo_conn[settings.MONGODB_DATABASE]
@app.on_event('shutdown')
async def shutdown_db_client():
    app.mongo_conn.close()
```

8. Define schemes @ `models/db_schemes/` for MongoDB, initially with any collection in MongoDB contain on `_id: UUID`, so add it optionally like

```
from pydantic import BaseModel, Field, validator
from typing import Optional
from bson.objectid import ObjectId # bson installed by default with motor
class Project(BaseModel):
    _id: Optional[ObjectId] # pydantic doesn't contain ObjectId obj,
                           # so define `class Config`
    project_id: str = Field(..., min_length=1)
    @classmethod
    @validator('project_id')
    def validate_project_id(cls, value:str):
        if not value.isalnum():
            raise ValueError('project_id has to be alphanumeric')
        # return super().validate(value)
        return value
    class Config:
        arbitrary_types_allowed = True
```

9. As we will need `db_client` & some `config` @ Data Models then we will define `BaseDataModel` as a parent like

```
from helpers.config import get_settings, Settings
class BaseDataModel:
    def __init__(self, db_client: object):
        self.db_client = db_client
        self.app_settings: Settings = get_settings()
```

10. We will give `db_client` for `Models` as

```
@data_router.post('/process/{project_id}')
async def process_endpoint(request: Request, project_id: str,
                           process_request: ProcessRequest):
    project_model = ProjectModel(db_client=request.app.db_client)
```

11. We will create json obj in `collection` and **we deal with `collection` with json object so we will convert it into obj and versa versa** as

```
async def create_chunk(self, chunk: DataChunk):
    result = await self.collection.insert_one(
        chunk.dict(by_alias=True, exclude_unset=True)
    ) # by_alias=True, exclude_unset=True to consider
      # `_id` in `_id: Optional[ObjectId] = Field(None, alias='_id')`
    chunk.id = result.inserted_id
    return chunk
```

12. To find some json in `collection` do

```
async def get_chunk(self, chunk_id: str):
    result = await self.collection.find_one({
        '_id': ObjectId(chunk_id),
    })
    return DataChunk(**result) if result else None
```

13. To insert bulk of objects for efficient do

```
async def insert_many_chunks(self, chunks: list, batch_size: int = 100):
    for i in range(0, len(chunks), batch_size):
        batch = chunks[i:i+batch_size]
```

```

        operation = [
            InsertOne(chunk.dict(by_alias=True, exclude_unset=True))
            for chunk in batch
        ]
        await self.collection.bulk_write(operation)
    return len(chunks)

```

14. To delete obj do

```

async def delete_chunks_by_project_id(self, project_id: ObjectId):
    result = await self.collection.delete_many({
        'chunk_project_id': project_id,
    })
    return result.deleted_count

```

15. To do pagination do

```

async def get_all_project(self, page: int = 1, page_size: int = 10):
    # count total number of documents
    total_documents = await self.collection.count_documents({})
    # calculate total number of pages
    total_pages = total_documents // page_size
    if total_documents % page_size > 0:
        total_pages += 1
    cursor = self.collection.find().skip( (page-1) * page_size )
        .limit(page_size)
    projects = []
    async for document in cursor:
        projects.append(Project(**document))
    return projects, total_pages

```

Code Improvement

1. Add `username` & `password` for data base @ `docker/.env`, use them @ `docker-compose.yml` and update `.env` with them like

```

MONGO_INITDB_ROOT_USERNAME=admin
MONGO_INITDB_ROOT_PASSWORD=admin

environment:
    - MONGO_INITDB_ROOT_USERNAME=${MONGO_INITDB_ROOT_USERNAME}
    - MONGO_INITDB_ROOT_PASSWORD=${MONGO_INITDB_ROOT_PASSWORD}

MONGODB_URL="mongodb://<username>:<password>@localhost:27007"

```

2. To create indeces for collection define indeces and assign them like

```

@classmethod
def get_indexes(cls):
    return [
        {
            'key': [
                ('asset_project_id', 1),    # 1 for Asc, -1 for Desc
            ],
            'name': 'asset_project_id_index_1',    # unique index name
            'unique': False,
        },
        {

```

```

        'key':[
            ('asset_project_id', 1),
            ('asset_name', 1),
        ],
        'name': 'asset_project_id_name_index_1',
        'unique': True,
    },
]

class ProjectModel(BaseDataModel):
    # Note that `__init__` hasn't to be async
    def __init__(self, db_client: object):
        super().__init__(db_client=db_client)
        self.collection = self.db_client[DataBaseEnum.COLLECTION_PROJECT_NAME.value]

    # `init_collection` has to be async, so it use mongodb fn
    async def init_collection(self):
        all_collections = await self.db_client.list_collection_names()
        if DataBaseEnum.COLLECTION_PROJECT_NAME.value not in all_collections:
            self.collection = self.db_client[DataBaseEnum
                .COLLECTION_PROJECT_NAME.value]
        indexes = Project.get_indexes()
        for index in indexes:
            await self.collection.create_index(
                index['key'],
                name=index['name'],
                unique=index['unique'],
            )

    # `init_collection` has to be called after `__init__` directly,
    # so we create thisss method
    @classmethod
    async def create_instance(cls, db_client: object):
        instance = cls(db_client)
        await instance.init_collection()
        return instance

```

1. To search by indexing do

```

async def get_all_project_assets(self, asset_project_id: str):
    return await self.collection.find({
        'asset_project_id': ObjectId(asset_project_id) if
        isinstance(asset_project_id, str) else asset_project_id,
    }).to_list(length=None)

```

LLM

1. define `factory design pattern` & `@ providers/` define supported LLM and their name in `LLMEnums` and global interface `@ LLMInterface` and do this structure

```

stores
├── __init__.py
├── llm
│   ├── __init__.py
│   └── LLMEnums.py

```

```

└─ LLMInterface.py
└─ providers
    └─ __init__.py
    └─ OpenAIProvider.py

```

```

from enum import Enum
class LLMEnums(Enum):
    OPENAI = "OPENAI"
    COHERE = "COHERE"

from abc import ABC, abstractmethod
class LLMInterface(ABC):
    @abstractmethod
    def set_generation_model(self, model_id:str):
        pass
    @abstractmethod
    def set_embedding_model(self, model_id: str, embedding_size: int):
        pass
    @abstractmethod
    def generate_text(self, prompt: str, max_output_tokens: int,
        temperature: float = None):
        pass
    @abstractmethod
    def embed_text(self, text: str, document_type: str):
        pass
    @abstractmethod
    def construct_prompt(self, prompt: str, role: str):
        pass

```

- Avoid `smell code` possible to make disaster
- With dealing with 3rd party (AI provider) enhancement your validation to metegate error and changing AI provider interface like

```

def embed_text(self, text: str, document_type: str):

    if not self.client:
        self.logger.error('Embedding model for OpenAI was not set')
        return None

    if not self.embedding_model_id:
        self.logger.error('Embedding model for OpenAI was not set')
        return None

    response = self.client.embeddings.create(
        model = self.embedding_model_id,
        input = text,
    )

    if not response or not response.data or len(response.data) == 0 or \
        not response.data[0].embedding:
        self.logger.error('Error while ebedding text with OpenAI')
        return None

    return response.data[0].embedding

```